

Week 03 - Lab 1

Game Entity Management with std::vector

Duration	2 hours
Lab Type	Guided / self-explanatory practical
Context	Game development - player scores and enemy data
Submission	No Submission

Learning Outcomes

- Create, populate and inspect std::vector objects.
- Use push_back(), pop_back(), front(), back(), at(), [] and erase().
- Explain the difference between size() and capacity().
- Experiment with reserve() and resize().
- Traverse and modify vector elements using iterators.
- Use find_if() with lambdas to search by condition.
- Remove defeated game entities using remove_if() and the erase-remove idiom.

Before you start

This lab is guided, but you should still type, compile and test each section yourself. The starter functions below are provided only to handle repeated display/formatting work so that you can focus on vectors and STL algorithms.

Starter Code - Add This at the Top of Your .cpp File

The following code is provided for you. You may copy it into your file before starting Part 1. It contains the Enemy structure plus three small helper functions used later in the lab.

```
#include <iostream>
#include <vector>
#include <string>
#include <algorithm>
#include <iomanip>

using namespace std;

struct Enemy
{
    int id;
    string name;
    int health;
    int score;
};

void printSection(const string& title)
{
    cout << "\n===== \n";
    cout << title << "\n";
    cout << "===== \n";
}

void displayIntVector(const vector<int>& values)
{
    cout << "[ ";
```

```

    for (int value : values)
        cout << value << " ";
    cout << "]\n";
}

void displayEnemies(const vector<Enemy>& enemies)
{
    cout << left
        << setw(8) << "ID"
        << setw(15) << "Name"
        << setw(10) << "Health"
        << setw(10) << "Score"
        << "\n";

    cout << string(43, '-') << "\n";

    for (const Enemy& e : enemies)
    {
        cout << left
            << setw(8) << e.id
            << setw(15) << e.name
            << setw(10) << e.health
            << setw(10) << e.score
            << "\n";
    }
}

```

What the helper functions do

- `printSection(title)`: prints a clear heading so the output for each part is easy to read.
- `displayIntVector(values)`: prints every integer in a vector on one line. Use it after `push_back()`, `pop_back()` and `resize()` to inspect the current contents.
- `displayEnemies(enemies)`: prints Enemy objects in aligned columns. You can call this whenever you need to inspect the current enemy list.
- `const vector<...>&` means the function reads the original vector without making a copy and without changing it.

Part 1 - Player Score Vector

Start inside `main()` with the vector below:

```
std::vector<int> scores {50, 80, 40, 90};
```

Task 1.1 - Display and inspect the scores

1. Use an index-based loop to display each value together with its index.
2. Print `size()` and `capacity()`. Remember: `size` is the number of actual elements; `capacity` is the amount of prepared storage.
3. Print the first element using `front()`, the last using `back()`, and the value at index 2 using `at(2)` or `[2]`.

Hint

An index-based loop normally starts at 0 and continues while the index is less than `scores.size()`.

Your output should clearly show Index 0: 50, Index 1: 80, Index 2: 40 and Index 3: 90.

Task 1.2 - Add, remove and modify

Perform these operations one at a time. After every operation call `displayIntVector(scores)`, then print the current size and capacity.

```
scores.push_back(70);
scores.push_back(100);
scores.pop_back();
```

Finally, change the value 80 to 85 using its index and display the vector again.

Checkpoint

If `size()` returns 5, write a comment listing all valid indices. Remember the last valid index is `size() - 1`.

Part 2 - Watching Capacity Change

Create an empty vector and add values one at a time:

```
std::vector<int> gameObjects;
// Add: 10, 20, 30, 40, 50, 60, 70, 80
```

After each `push_back()`, print the value added, `size()` and `capacity()`. You are observing when the vector decides it needs a larger block of storage.

Important

The exact capacity sequence may differ between compilers/libraries. Do not assume the vector always doubles its capacity. Record what your program actually reports.

Add short comments answering:

- At which additions did capacity change?
- Why can capacity be larger than size?
- Why should you not assume a fixed growth rule?

Part 3 - `reserve()`: Planning Ahead

```
std::vector<int> enemyIDs;
enemyIDs.reserve(20);
```

Before adding any IDs, print `size()` and `capacity()`. Then add enemy IDs 101 to 110 using `push_back()`. After each insertion, print the new size and capacity.

Think before coding

`reserve(20)` does not create 20 IDs. It only asks the vector to prepare storage for at least 20 elements. `size()` should still be 0 until you actually push values into the vector.

Complete these comments in your program:

```
// Q1: Did reserve(20) create 20 elements?
// Answer:
// Q2: Why did size remain 0 immediately after reserve(20)?
// Answer:
// Q3: What benefit did reserve(20) provide while adding IDs?
// Answer:
```

Part 4 - reserve() vs resize()

Create a new vector named slots and test reserve first:

```
std::vector<int> slots;
slots.reserve(5);
```

Print the size and capacity. Then run:

```
slots.resize(5);
```

Print size, capacity and the actual elements by calling displayIntVector(slots).

What to look for

After reserve(5), the vector has prepared space but contains no actual elements. After resize(5), five int elements exist. For int, the newly created elements will normally appear as 0.

Explain the difference in comments:

```
// reserve() means:
// resize() means:
```

Part 5 - Move to Real Game Objects

Now use the Enemy structure already provided in the starter code. Create:

```
std::vector<Enemy> enemies;
```

Add the following objects using push_back(). The first example is shown; add the rest yourself.

```
enemies.push_back({101, "Goblin", 75, 120});
```

ID	Enemy	Health	Score
101	Goblin	75	120
102	Orc	90	300
103	Skeleton	15	220
104	Mage	40	550
105	Troll	100	450
106	Rogue	18	700

When all six enemies have been added, call displayEnemies(enemies). Check that the values appear in the expected columns.

Part 6 - Traverse with an Iterator

For this part, do not use an index loop. Traverse the vector using an iterator:

```
for (auto it = enemies.begin(); it != enemies.end(); ++it)
{
    // Display the current Enemy using it->id, it->name, ...
}
```

Then create an iterator pointing to the first enemy, move it once using ++it so that it points to the second enemy, and change that enemy's health from 90 to 70.

Hint

When an iterator points to an Enemy object, use `it->health`, `it->name`, `it->score` and `it->id` to access its fields. `*it` represents the whole Enemy object at the current position.

Write one sentence as a comment explaining what `*it` represents.

Part 7 - Insert and Erase a Boss

```
Enemy boss {999, "Dragon", 200, 1500};
```

- Insert boss at `enemies.begin()`.
- Call `displayEnemies(enemies)` and observe that all original elements have shifted one position.
- Remove the first element using `erase(enemies.begin())`.
- Display the vector again and confirm the original ordering is restored.

Iterator warning

After `insert()`, `erase()`, or a `push_back()` that causes reallocation, old iterators may no longer be valid. Do not blindly reuse an iterator saved before the change.

Part 8 - Search by Condition with `find_if()`

Use `std::find_if()` to locate the first enemy whose health is below 20. The search should start at `begin()` and stop at `end()`. The lambda should return true only for enemies that satisfy the condition.

```
auto criticalEnemy = std::find_if(
    enemies.begin(),
    enemies.end(),
    [](const Enemy& e)
    {
        // return true when health is below 20
    }
);
```

Before using the result, check that `criticalEnemy != enemies.end()`. If found, display the enemy name, health and score.

Remember

`find_if()` stops at the first element for which the lambda returns true. It does not return every matching enemy.

Part 9 - Change Only the Lambda

Reuse the same `find_if()` structure three times. Keep `begin()`, `end()` and the general shape of the code unchanged; only change the condition inside the lambda.

- Find the first enemy whose score is greater than 500.
- Find the first enemy whose health is at least 80.
- Find the first enemy whose health is below 30 AND whose score is greater than 500.

Logic hint

For the final search, both conditions must be true at the same time. In C++, logical AND is written as `&&`.

Part 10 - Search Using Player Input

Ask the user to enter an enemy ID. Unlike the previous lambdas, this search needs a value that was entered outside the lambda. Capture that value so the lambda can compare it with `e.id`.

```
int searchID;
cout << "Enter enemy ID to search: ";
cin >> searchID;

auto enemyByID = std::find_if(
    enemies.begin(),
    enemies.end(),
    [searchID](const Enemy& e)
    {
        // compare e.id with searchID
    }
);
```

If found, print ID, name, health and score. Otherwise print: Enemy does not exist.

Why [searchID]?

The capture list allows the lambda to use the `searchID` variable that was created outside the lambda.

Part 11 - Remove Defeated Enemies

Change the health of at least three enemies so they are defeated. An enemy is defeated when `health <= 0`.

Then use the erase-remove pattern. Do not remove enemies one-by-one with repeated `erase()` calls.

```
auto newEnd = std::remove_if(
    enemies.begin(),
    enemies.end(),
    [](const Enemy& e)
    {
        // return true for defeated enemies
    }
);

enemies.erase(newEnd, enemies.end());
```

Before cleanup, store `enemies.size()`. After cleanup, compare the new size with the old size to calculate how many enemies were removed.

What is happening?

`remove_if()` rearranges the elements we want to keep and returns a new logical end. `erase()` then physically removes the unwanted tail from the vector.

Part 12 - Guided Challenge: Emergency Healing

After cleanup, find the first living enemy whose health is below 25. This requires two conditions: the enemy must still be alive and its health must be below 25.

If a suitable enemy is found:

- Store its old health in a separate variable.
- Increase its health by 20.
- If the new health is greater than 100, set it back to 100.
- Display the enemy name, old health and new health.

If no suitable enemy exists, display: No enemy currently needs emergency healing.

Logical hint

A living enemy has health > 0. Combine this with health < 25 using && inside the lambda.

Final Checklist

- The program compiles without errors.
- The provided helper functions are used where appropriate.
- The program demonstrates vector operations, size/capacity, reserve/resize and iterators.
- find_if() is used for condition-based searching.
- remove_if() + erase() is used for cleanup.
- Your code is indented and includes short comments for the conceptual questions.
- Submit one .cpp file only.

End of Lab

By the end of this lab you should be able to use std::vector as a practical game-data container, inspect its storage behaviour, traverse it with iterators, search it using lambdas, and remove defeated entities using STL algorithms.